

M02 / STUDY COMPANION

M02 Quick Revision Book

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 4

How to revise 4

DE02-01 | Database, table, row, column and query mental model 4

60-second recall 4

Recall task 4

DE02-02 | SELECT, aliases and LIMIT 4

60-second recall 4

Recall task 4

DE02-03 | WHERE and comparison operators 5

60-second recall 5

Recall task 5

DE02-04 | AND, OR, NOT and parentheses 5

60-second recall 5

Recall task 5

DE02-05 | ORDER BY, DISTINCT and result inspection 5

60-second recall 5

Recall task 5

DE02-06 | NULL, IS NULL and COALESCE 6

60-second recall 6

Recall task 6

DE02-07 | Arithmetic expressions and safe calculations 6

60-second recall 6

Recall task 6

DE02-08 | Aggregate functions 6

60-second recall 6

Recall task 6

DE02-09 | GROUP BY and grouped grain 7

60-second recall 7

Recall task 7

DE02-10 | HAVING after aggregation 7

60-second recall 7

Recall task 7

DE02-11 | CASE for business categories 7

60-second recall 7

Recall task 7

DE02-12 | String and date basics 8

60-second recall 8

Recall task 8

DE02-13 | Simple subqueries and IN/EXISTS awareness 8

60-second recall 8

Recall task 8

DE02-14 | Debugging and validation workflow 8

60-second recall 8

Recall task 8

Read this first

QUICK REVISION BOOK

Use this only after you have studied the learner book. Each lesson keeps the canonical language and core proof, plus one recall task. This is not a replacement for hands-on practice.

Coverage: 14 canonical lessons

Route: learner book -> practice -> revision -> closed-book test -> protected answers -> changed retry.

How to revise

Read the lesson card, close the page, explain it in your own words, then answer the recall task. If recall is weak, return to that lesson only - not the entire module.

DE02-01 | Database, table, row, column and query mental model

60-second recall

A query is a request; the result is its answer at execution time. `course.sales` names schema and table. `SELECT` chooses displayed fields, `FROM` chooses input, and `ORDER BY` requests their order. SQL text, stored table and displayed result are three different things. Teach it back: point to each of those three objects in your own working session.

[Refresh source: Learn PostgreSQL Tutorial - Full Course for Beginners](#)

Recall task

Use `course.retry_sales`. Request `order_id`, `quantity` and `unit_price` arranged by `order_id`. Explain why the order ID and quantity have different roles even though both may contain digit characters.

DE02-02 | SELECT, aliases and LIMIT

60-second recall

`AS` names an output; `LIMIT` caps a result. Use `ORDER BY` whenever which limited rows appear matters. `SELECT item` reads a column, while `SELECT 'item'` returns literal text. A preview is not a row-count check, and an alias is not a permanent table edit.

[Refresh source: Learn PostgreSQL Tutorial - Full Course for Beginners](#)

Recall task

From `course.retry_sales`, return the last two IDs in descending order. Use the output label reference and retain item. `DESC` after `order_id` asks for descending order. Explain whether a second new query must use reference or `order_id` to select the original ID column.

DE02-03 | WHERE and comparison operators

60-second recall

WHERE accepts individual input rows when its condition is true. Choose both the correct field and the correct operator. Check an exact boundary value, use single quotes for text, and use explicit year-month-day date literals. A successful execution can still answer the wrong business question.

[Refresh source: Learn PostgreSQL Tutorial - Full Course for Beginners](#)

Recall task

From `course.retry_sales`, return rows ordered before November 2, 2026, with ID and date. Explain whether a November 2 record belongs in "before November 2."

DE02-04 | AND, OR, NOT and parentheses

60-second recall

Translate the sentence into smaller tests before typing. AND requires both, OR permits either, and NOT reverses a known truth value. Parentheses make the grouping explicit. Validate membership with row IDs, not just an output count. Teach it back by explaining why 0009 leaked through the deliberately wrong guided query.

[Refresh source: PostgreSQL Tutorial Full Course 2022](#)

Recall task

From `course.retry_sales`, return records that are not Paper and have quantity at least two. State which two rules each returned row satisfies.

DE02-05 | ORDER BY, DISTINCT and result inspection

60-second recall

ORDER BY belongs to a result request. Add a stable tie-breaker when exact ordering matters. DISTINCT compares the whole selected combination, not just its first column. Unique products and unique sales have different grains; fewer output rows can be correct for one question and wrong for another.

[Refresh source: Learn PostgreSQL Tutorial - Full Course for Beginners](#)

Recall task

Return distinct item and unit_price combinations from `course.retry_sales`. Then request the first two records under descending quantity and ascending order_id. Explain how the second key resolves the tied quantity of three.

DE02-06 | NULL, IS NULL and COALESCE

60-second recall

Use IS NULL for missingness. Unknown, false, empty text and zero are different states. WHERE retains true, not unknown. COALESCE applies an explicit fallback; preserve the raw value and state any assumption. Teach it back: why does NOT is_paid exclude unknown payments?

[Refresh source: SQL for Data Analytics - Learn SQL in 4 Hours](#)

Recall task

Use course.retry_sales. Return IDs whose customer_id is missing. Separately return IDs whose discount_pct is missing. Explain why those need not be the same record.

DE02-07 | Arithmetic expressions and safe calculations

60-second recall

Name measures with their meaning and unit. Check operator order and fee grain. Numeric types affect division. NULL arithmetic stays unknown; COALESCE needs a declared rule. NULLIF protects a ratio from a zero denominator while preserving its undefined meaning.

[Refresh source: Learn PostgreSQL Tutorial - Full Course for Beginners](#)

Recall task

Calculate gross amounts for course.retry_sales. Separately compute a return rate using 3 returned units and 0 sold units, preserving an undefined denominator with NULLIF. Explain why zero percent would misrepresent that case.

DE02-08 | Aggregate functions

60-second recall

Choose the aggregate by the question. COUNT(*) counts rows; COUNT(field) counts non-null values, including FALSE. A whole-selection aggregate returns one summary row even for an empty selection. SUM over no input is NULL. Reconcile totals and explain the denominator of every average.

[Refresh source: Learn PostgreSQL Tutorial - Full Course for Beginners](#)

Recall task

Summarise course.retry_sales with total rows, known-payment rows, total quantity, gross total and mean sale amount. Explain why total quantity is not the denominator of the mean sale amount.

DE02-09 | GROUP BY and grouped grain

60-second recall

GROUP BY changes grain. State the complete grouping key and use named aggregates. Adding another grouping field changes the question. Check group membership, source counts and subtotal reconciliation. A null grouping bucket does not establish a shared real-world identity.

[Refresh source: Learn PostgreSQL Tutorial - Full Course for Beginners](#)

Recall task

Group `course.retry_sales` by `ordered_on`. Return `sale_rows`, `units` and `gross_total` for each day. Reconcile all three measures to the source.

DE02-10 | HAVING after aggregation

60-second recall

WHERE filters rows before grouping; HAVING filters completed groups. Translate each condition into the stage at which its required value exists. Inspect the unfiltered grouped result before blaming HAVING. PostgreSQL's HAVING should use the aggregate expression here rather than its SELECT alias.

[Refresh source: Learn PostgreSQL Tutorial - Full Course for Beginners](#)

Recall task

From `course.retry_sales`, keep only groups with at least two sales records per category. Return category, count and gross total. Explain why `COUNT(*)` belongs in HAVING here.

DE02-11 | CASE for business categories

60-second recall

CASE returns one value per row from the first true branch. Preserve missingness with an explicit rule. Test overlapping thresholds in the intended order, include boundary cases, and reconcile mutually exclusive categories. A friendly label is useful only if its rule is correct.

[Refresh source: Learn SQL Beginner to Advanced in Under 4 Hours](#)

Recall task

In `course.retry_sales`, define High as gross amount at least 75, Mid as at least 30 but below 75, and Low as below 30. Return ID and group, then identify the exact-boundary records that test your rule.

DE02-12 | String and date basics

60-second recall

Keep raw and cleaned text together. TRIM handles surrounding ordinary spaces, not arbitrary spelling or all whitespace. Date arithmetic uses calendar values; EXTRACT retrieves components. Use explicit typed dates and an inclusive-start/exclusive-end filter. Learn the idea from the video and use the book's PostgreSQL syntax.

[Refresh source: Learn PostgreSQL Tutorial - Full Course for Beginners](#)

Recall task

For R203 in course.retry_sales, display the raw item, uppercase item, month number and date seven days later. Explain whether uppercase changes the stored value.

DE02-13 | Simple subqueries and IN/EXISTS awareness

60-second recall

Run the inner query separately when learning. A scalar subquery supplies one value, IN tests membership, and EXISTS tests whether any match exists. State the threshold population. Beware nullable NOT IN lists; do not disguise a many-row error with an arbitrary LIMIT.

[Refresh source: Learn PostgreSQL Tutorial - Full Course for Beginners](#)

Recall task

Find course.retry_sales records at or above the retry table's average gross amount. State the threshold and explain whether EXISTS could replace that scalar threshold without changing the question.

DE02-14 | Debugging and validation workflow

60-second recall

Write the contract first: source, eligible rows, output grain, measure and final conditions. Repair syntax before meaning, then validate membership, boundaries and reconciliation at the correct scope. Keep the first attempt and changed retry separate. You are ready for M03 when you can build these M02 queries from a blank editor, explain their results and repair a wrong condition without guessing.

[Refresh source: SQL for Data Analytics - Learn SQL in 4 Hours](#)

Recall task

For course.retry_sales, report gross amount by payment category with Paid, Unpaid and Unknown all preserved. Reconcile the category row counts and amounts to the source. You may repeat a CASE in GROUP BY; do not use a SELECT alias in WHERE. Explain what would go wrong if Unknown were put into Unpaid.